

uMayFinish, pEchoVar, and pDeadManPost

Mission completion, echoing, and dead-man posting utilities.

Included MIT MOOS-IvP PDF sources:

- app_umayfinish.pdf
- app_pechovar.pdf
- app_pdeadmanpost.pdf

uMayFinish: A command-line MOOS Tool for Waiting for Mission Completion

January 2025

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139
project-pavlab/appdocs/app_umayfinish

1	Overview	1
2	Configuration Parameters for uMayFinish	2
3	Publications and Subscriptions for uMayFinish	3
3.1	Variables Published by uMayFinish	3
3.2	Variables Subscribed for by uMayFinish	3
3.3	Command Line Usage of uMayFinish	4
4	Terminal and AppCast Output	4

1 Overview

uMayFinish is typically a terminal-launched MOOS app launched within a shell script, e.g., **xlaunch.sh**. It will connect to a MOOS community and monitor for a completion event or timeout based on **DB_UPTIME**. When completed, it simply exits. Presumably to allow the executing script to proceed to a next phase. For example a script could proceed to bring down the MOOS community for the mission it was monitoring.

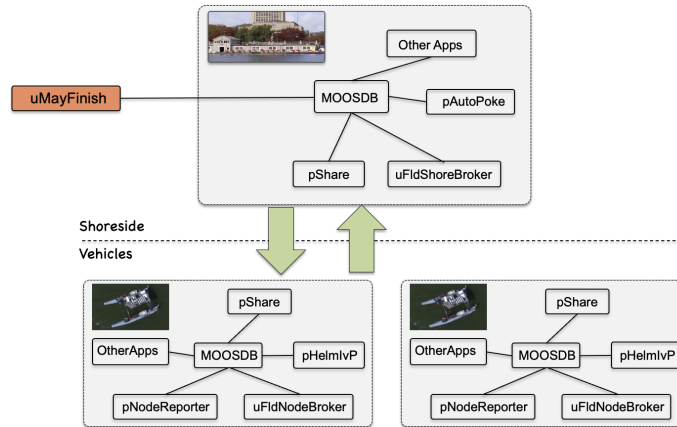


Figure 1: **Typical uMayFinish Topology:** The **uMayFinish** app is typically not part of the initial MOOS community of either the shoreside or vehicles. Typically, shortly after everything is launched, the **uMayFinish** app is launched and connects to the shoreside MOOSDB. It waits for either a time limit to be reached, or a specified MOOS variable to have a specified value. Then the app disconnects. The disconnection may be followed by an action to shut down all MOOS communities.

Consider the following pseudo-code snippet:

```
pAntler mission.moos &
uMayFinish --max_time=600 mission.moos
ktm
```

In the first line, the mission is launched. It is launched in the background, so control proceeds immediately to the next line. Then **uMayFinish** launches and connects to the same MOOSDB since it is invoked with the same mission file. It is *not* launched in the background, so control will stop here until the **uMayFinish** completes. After 10 minutes (600 seconds), it will disconnect from the MOOSDB, and exit. Control passes to the next line which invokes **ktm** (kill-the-moos), to bring down the entire mission.

Something similar is used in **xlaunch.sh**, which is meant to be a general way for auto-launching missions with provisions for time-conditioned or event-conditioned exiting. In the above snippet, the maximum time was passed on the command line. The maximum time can also be specified in the **uMayFinish** configuration file in the mission file, as described in Section 2.

2 Configuration Parameters for uMayFinish

The **uMayFinish** application may be configured with a configuration block within a MOOS mission file, typically with a **.moos** file suffix. The following parameters are defined for **uMayFinish**.

Listing 2.1: Configuration Parameters for uMayFinish.

finish_var: A MOOS variable.
finish_val: A prescribed value for the MOOS variable.

`max_db_uptime`: The number of seconds, found in the MOOSDB published variable `DB_UPTIME`, after which this app will exit.

An Example MOOS Configuration Block

An example MOOS configuration block may be obtained from the command line with the following:

```
$ uMayFinish --example or -e
```

Listing 2.2: Example configuration of the uMayFinish application.

```
1 =====
2 uMayFinish Example MOOS Configuration
3 =====
4
5 ProcessConfig = uMayFinish
6 {
7     AppTick    = 4
8     CommsTick  = 4
9
10    finish_var = MISSION_EVALUATED // Default
11    finish_val = true               // Default
12
13    max_db_uptime = 3600            // Default is -1 (off)
14 }
```

3 Publications and Subscriptions for uMayFinish

The interface for `uMayFinish`, in terms of publications and subscriptions, is described below. This same information may also be obtained from the terminal with:

```
$ uMayFinish --interface or -i
```

3.1 Variables Published by uMayFinish

- `APPCAST`: Contains an appcast report identical to the terminal output. Appcasts are posted only after an appcast request is received from an appcast viewing utility.

3.2 Variables Subscribed for by uMayFinish

The `uMayFinish` application subscribes to the following MOOS variables:

- `APPCAST_REQ`: A request to generate and post a new appcast report, with reporting criteria, and expiration.
- `DB_UPTIME`: The number of seconds since the MOOSDB has been launched. If a mission has been auto-started shortly after launch (e.g., using `pAutoPoke`), then this value is a good approximation for measuring how long the mission has been running. Note, this subscription will only be made if the configuration parameter `max_db_uptime` is set. This param is set to `-1` by default, meaning there is no maximum time.

Node that `uMayFinish` will also subscribe for the variable named in the configuration parameter `finish_var`, if provided.

3.3 Command Line Usage of uMayFinish

The `uMayFinish` application is typically launched with `pAntler`, along with a group of other modules. However, it may be launched separately from the command line. The command line options may be shown by typing:

```
$ uMayFinish --help or -h
```

Listing 3.3: Command line usage for the uMayFinish tool.

```
1 Usage: uMayFinish file.moos [OPTIONS]
2
3 Options:
4   --alias=<ProcessName>
5       Launch uMayFinish with the given process
6       name rather than uMayFinish.
7   --example, -e
8       Display example MOOS configuration block
9   --help, -h
10      Display this help message.
14  --interface, -i
15      Display MOOS publications and subscriptions.
16  --version, -v
17      Display the release version of uMayFinish.
18  --max_time=1200
19      Maximum DB_UPTIME before exiting.
20  --web, -w
21      Open browser to: https://oceanai.mit.edu/ivpman/apps/uMayFinish
22
23 Note: If argv[2] is not of one of the above formats
24       this will be interpreted as a run alias. This
25       is to support pAntler launching conventions.
```

4 Terminal and AppCast Output

Some useful information is published by `uMayFinish` to the terminal on every iteration. An example is shown in Listing 4 below. This application is also appcast enabled, meaning its reports are published to the MOOSDB and viewable from any uMAC application or `pMarineViewer`. The counter on the end of line 2 is incremented on each iteration of `uMayFinish`, and also serves as a heartbeat indicator. The "0/0" also on line 2 indicates there are no configuration or run warnings detected.

Listing 4.4: Example terminal or appcast output for uMayFinish.

```
1 =====
2 uMayFinish shoreside                                0/0(67)
3 =====
4 Config:
5   Finish Var: MISSION_EVALUATED
6   Finish Val: true
```

```

7   Max DBTime: -1
8
9   State:
10  DB_UPTIME: n/a
11  MISSION_EVALUATED:

```

Lines 4-7 of the output show the user configuration. Lines 10 and 11 show the current state of the app. Note the **DB_UPTIME** output on line 10 indicates "n/a" since no max time was set for this launch (line 7).

When the exit criteria is met, the appcast output may change as below. Note the only change is on line 11, where the empty value for the current value of **MISSION_EVALUATED** in the listing above, is replaced with **MISSION_EVALUATED: true** in the listing below. Since this satisfies the exit criteria, this will be the very last appcasting output to either the terminal or seen in **pMarineViewer**.

*Listing 4.5: Example output as **uMayFinish** finishes..*

```

1  =====
2  uMayFinish shoreside                                0/0(341)
3  =====
4  Config:
5    Finish Var: MISSION_EVALUATED
6    Finish Val: true
7    Max DBTime: -1
8
9  State:
10  DB_UPTIME: n/a
11  MISSION_EVALUATED: true

```

When **uMayFinishApp** exits due to satisfying the finish variable criteria, it will return with a value of 0. When **uMayFinishApp** exits due to exceeding a maximum time limit, the exit value is 1 instead.

pEchoVar: Re-publishing Variables Under a Different Name

June 2018

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139
`project-pavlab/appdocs/app_pechovar`

1	Overview	1
2	Using pEchoVar	1
2.1	Configuring Echo Mapping Events	2
2.2	Configuring Flip Mapping Events	2
2.3	Applying Conditions to the Echo and Flip Operation	3
2.4	Holding Outgoing Messages Until Conditions are Met	4
2.5	Limiting the Echo Posting Frequency to the AppTick Setting	4
3	Configuring for Vehicle Simulation with pEchoVar	4
4	Configuration Parameters for pEchoVar	5
5	Publications and Subscriptions for pEchoVar	5
5.1	Variables Posted by pEchoVar	6
5.2	Variables Subscribed for by pEchoVar	6
6	Terminal and AppCast Output	6

1 Overview

The pEchoVar application is a lightweight process that runs without any user interaction for "echoing" the posting of specified variable-value pairs with a follow-on posting having different variable name. For example the posting of `F00=5.5` could be echoed such that `BAR=5.5` immediately follows the first posting. The motivation for this tool was to convert, for example, a posting such as `GPS_X` to become `NAV_X`. The former is the output of a particular device, and the latter is a de facto standard for representing the vehicle's longitudinal position in local coordinates.

2 Using pEchoVar

Configuring `pEchoVar` minimally involves the specification of one or more `echo` or `flip` mapping events. It may also optionally involve specifying one or more logic conditions that must be met before mapping events are posted.

2.1 Configuring Echo Mapping Events

An *echo event mapping* maps one MOOS variable to another. Each mapping requires one line using the `echo` configuration parameter of the form:

```
echo = <MOOSVar> -> <MOOSVar>
```

The source `<MOOSVar>` and target `<MOOSVar>` components are case sensitive since they are MOOS variables. A source variable can be echoed to more than one target variable. If the set of lines forms a cycle, this will be detected and `pEchoVar` post a configuration and run warning and cease to perform any function whatsoever. An example configuration is given in Listing 1.

Listing 2.1: An example pEchoVar configuration block.

```
1  //-----
2  // pEchoVar configuration block
3
4  ProcessConfig = pEchoVar
5  {
6      AppTick    = 20
7      CommsTick  = 20
8
9      echo = GPS_X          -> NAV_X
10     echo = GPS_Y          -> NAV_Y
11     echo = COMPASS_HEADING -> NAV_HEADING
12     echo = GPS_SPEED      -> NAV_SPEED
13 }
```

2.2 Configuring Flip Mapping Events

The `pEchoVar` application can be used to "flip" a variable rather than doing a simple echo. A flipped variable, like an echoed variable, is one that is republished under a different name, but a flipped variable parses the contents of a string comprised of a series of `variable=value` comma-separated pairs, and republishes a portion of the series under the new variable name. For example, the following string,

```
ALPHA = "xpos=23, ypos=-49, depth=20, age=19.3, certainty=1"
```

may be flipped to publish the below new string, with the fields `xpos`, `ypos`, and `depth` replaced with `x`, `y`, `vehicle_depth` respectively.

```
BRAVO = "x=23, y=-49, vehicle_depth=20"
```

The above "flip relationship" is configured with the `flip` configuration parameter with the following form:

```
flip:<key>    = source_variable = <variable>
flip:<key>    = dest_variable   = <variable>
flip:<key>    = source_separator = <separator>
```

```

flip:<key>    = dest_separator    = <separator>
flip:<key>    = filter            = <variable>=<value>
flip:<key>    = component         = <old-field> -> <new-field>
flip:<key>    = component         = <old-field> -> <new-field>

```

The relationship is distinguished with a <key>, and several components. The `source_variable` and `dest_variable` components are mandatory and must be different. The `source_separator` and `dest_separator` components are optional with default values being the string ",". Fields in the source variable will only be included in the destination variable if they are specified in a component mapping <old-field> -> <new-field>. The example configuration in Listing 2 implements the above described example flip mapping. In this case only postings that satisfy the further filter, `certainty=1`, will be posted.

Listing 2.2: An example pEchoVar configuration block with flip mappings.

```

1  //-----
2  // pEchoVar configuration block
3
4  ProcessConfig = pEchoVar
5  {
6      AppTick    = 10
7      CommsTick  = 10
8
9      flip:1     = source_variable = ALPHA
10     flip:1     = dest_variable   = BRAVO
11     flip:1     = source_separator = ,
12     flip:1     = dest_separator  = ,
13     flip:1     = filter          = certainty=1
14     flip:1     = component       = ypos -> y
15     flip:1     = component       = xpos -> x
16 }

```

Some caution should be noted with flip mappings - the current implementation does not check for cycles, as is done with echo mappings.

2.3 Applying Conditions to the Echo and Flip Operation

The execution of the mappings configured in `pEchoVar` may be configured to depend on one or more logic conditions. If conditions are specified in the configuration block, all specified logic conditions must be met or else the posting of echo and flip mappings will be suspended. The logic conditions are configured with the `condition` parameter as follows:

```

condition = <logic-expression>

```

The <logic-expression> syntax is described in the Appendix document on logic utilities, and may involve the simple comparison of MOOS variables to specified literal values, or the comparison of MOOS variables to one another. If a `condition` parameter is specified, `pEchoVar` will automatically subscribe to all MOOS variables used in the condition expressions.

2.4 Holding Outgoing Messages Until Conditions are Met

If the conditions are not met, all incoming mail messages that would otherwise result in an echo or flip posting, are held. When or if the conditions are met at some point later, those mail messages are processed in the order received and echo and flip mappings may be posted en masse. However, if several mail messages for a given MOOS variable are received and stored while conditions are unmet, only the latest received mail message for that variable will be processed. As an example, consider `pEchoVar` configured with the below two lines:

```
echo      = FOO -> BAR
condition = DEGREES <= 32
```

If the condition is not met for some period of time, and the following mail were received during this interval: `FOO="apples"`, `FOO="pears"`, `FOO="grapes"`, followed by `DEGREES=30`, then `pEchoVar` would post `BAR="grapes"` immediately on the very iteration that the `DEGREES=30` message was received. Note that `BAR="apples"` and `BAR="pears"` would never be posted. This is to help ensure that the `pEchoVar` memory doesn't grow unbounded by holding onto all mail while conditions are unmet.

The user may alternatively configure `pEchoVar` to *not* hold incoming mail messages when or if it is in a state where its logic conditions are not met. This can be done with the `hold_messages` parameter:

```
hold_messages = false    // The default is true
```

When configured this way, upon meeting the specified logic conditions, `pEchoVar` will begin processing echo and flip mappings when or if new mail messages are received relevant to the mappings. In the above example, once `DEGREES=30` is received by `pEchoVar`, nothing would be posted until new incoming mail on the variable `FOO` is received (not even `BAR="grapes"`).

2.5 Limiting the Echo Posting Frequency to the AppTick Setting

By default, when the conditions are met, an echo posting is made once for each incoming piece of mail related that echo mapping. If `FOO` is echo mapped to `BAR`, and if 40 pieces of incoming mail for `FOO` are received on one iteration, 40 postings are made to `BAR` on that iteration. Instead one may wish that, on each iteration where there is posting ready for `BAR`, that only the latest value for `BAR` be made. This may be arranged with the `echo_latest_only` parameter:

```
echo_latest_only = true    // The default is false
```

In this case, the frequency of postings to `BAR` and all other echo mappings will occur at most at a frequency equal to the `AppTick` setting.

3 Configuring for Vehicle Simulation with pEchoVar

When in simulation mode with `uSimMarine`, the navigation information is generated by the simulator and not the sensors such as GPS or compass as indicated in lines 9-12 in Listing 1. The simulator instead produces `USM_*` values which can be echoed as `NAV_*` values as shown in Listing 3.

Listing 3.3: An example pEchoVar configuration block during simulation.

```
1 //-----
2 // pEchoVar configuration block (for simulation mode)
3
4 ProcessConfig = pEchoVar
5 {
6     AppTick    = 20
7     CommsTick  = 20
8
9     echo = USM_X      -> NAV_X
10    echo = USM_Y      -> NAV_Y
11    echo = USM_HEADING -> NAV_HEADING
12    echo = USM_SPEED   -> NAV_SPEED
13 }
```

Note in more recent versions of `uSimMarine` the simulator output may be changed to have a `NAV_` prefix by setting `prefix = NAV_`, obviating the use of `pEchoVar` configured as above.

4 Configuration Parameters for pEchoVar

The following parameters are defined for `pEchoVar`. A more detailed description is provided in other parts of this section. Parameters having default values are indicated so.

Listing 4.4: Configuration Parameters for pEchoVar.

- `echo`: A mapping from one MOOS variable to another constituting an echo. Section 2.1.
- `echo_latest_only`: If true, only the latest value of variable will be echoed on each iteration, even if several pieces of incoming mail have been received since the last posting. Legal values: true, false. The default is false. Section 2.1.
- `condition`: A logic condition that must be met or all echo and flip publications are held. Section 2.3.
- `flip`: A description of how components from one variable are re-posted under another MOOS variable. Section 2.2.
- `hold_messages`: If true, messages are held when conditions are not met for later processing when logic conditions are indeed met. Legal values: true, false. The default is true. Section 2.3.

5 Publications and Subscriptions for pEchoVar

The interface for `pEchoVar`, in terms of publications and subscriptions, is described below. This same information may also be obtained from the terminal with:

```
$ pEchoVar --interface or -i
```

5.1 Variables Posted by pEchoVar

- **APPCAST**: Contains an appcast report identical to the terminal output. Appcasts are posted only after an appcast request is received from an appcast viewing utility. Section 6.
- **<USER-DEFINED>**: Any MOOS variable specified in either the **echo** or **flip** config parameters.

5.2 Variables Subscribed for by pEchoVar

- **APPCAST_REQ**: A request to generate and post a new appcast report, with reporting criteria, and expiration.
- **<USER-DEFINED>**: Any MOOS variables found in the antecedent of either the **echo** or **flip** mappings. It will also subscribe for any MOOS variable found in any of its logic conditions.

6 Terminal and AppCast Output

The **pEchoVar** application produces some useful information to the terminal on every iteration of the application. An example is shown in Listing 5 below. This application is also appcast enabled, meaning its reports are published to the MOOSDB and viewable from any uMAC application or pMarineViewer. See the documentation on uMAC viewing utilities for more on appcasting and viewing appcasts. The counter on the end of line 2 in parentheses is incremented on each iteration of **pEchoVar**, and serves a bit as a heartbeat indicator. The "0/0" also on line 2 indicates there are no configuration or run warnings detected.

Listing 6.5: Example terminal or appcast output for pEchoVar.

```
1  =====
2  pEchoVar alpha                                0/0(81)
3  =====
4  conditions_met:   true
5  hold_messages:   true
6  echo_latest_only: false
7
8  =====
9  Echoes: (5)
10 =====
11
12 Source          Dest          Hits  Posts
13 -----
14 NAV_X           --> NAV_XX           324   324
15 NAV_X           --> NAV_XPOS          324   324
16 NAV_Y           --> NAV_YY           324   324
17 NAV_Y           --> NAV_YPOS          324   324
18 NAV_SPEED       --> NAV_SPEED_ALT    324   324
19
20 =====
21 Flips: (1)
22 =====
23
24 Key  Hits  Source          Dest  Src  Dest  Old  New
25 ---  ---  -----  -----  ---  ---  ---  ---
26
```

```

27 1    162  NODE_REPORT_LOCAL  FOOBAR  ,    #    type:kayak  X    xpos
28 1    162  NODE_REPORT_LOCAL  FOOBAR  ,    #    type:kayak  Y    ypos

```

Lines 4 indicates whether or not any specified logic conditions have been met. This line will also read `true` even if no logic conditions were provided. Lines 5 and 6 simply confirm the user's settings for the `hold_messages` and `echo_latest_only` parameters discussed in Sections 2.4 and 2.5 respectively.

Lines 12-18 convey the configured echo mappings, one for each line. At the end of each line, the *Hits* column shows the number of incoming mails received for that variable. The number of times it is echoed, or re-posted is shown under the *Posts* column. When `echo_latest_only` is false, these numbers should match. Lines 20-28 convey the configure flips. A single flip configuration, identified by its key, may have several lines, as in this example. Here the only difference between lines 27 and 28 are the flip components. One maps X to `xpos`, and the other maps Y to `ypos`.

The terminal or appcast output shown in Listing 5 above may be seen first hand by running the Alpha example mission. The below configuration block in Listing 6 corresponds to the above appcast output. The user just needs to add `pEchoVar` to the Antler launch list.

Listing 6.6: Example pEchoVar configuration from the Alpha example mission.

```

1  ProcessConfig = pEchoVar
2  {
3    AppTick = 1
4    CommsTick = 1
5
6    echo = NAV_X      -> NAV_XX
7    echo = NAV_X      -> NAV_XPOS
8    echo = NAV_Y      -> NAV_YY
9    echo = NAV_Y      -> NAV_YPOS
10   //echo = NAV_YY    -> FOOBAR
11   //echo = FOOBAR    -> NAV_Y
12   echo = NAV_SPEED -> NAV_SPEED_ALT
13
14   FLIP:1    = source_variable = NODE_REPORT_LOCAL
15   FLIP:1    = dest_variable  = FOOBAR
16   FLIP:1    = source_separator = ,
17   FLIP:1    = dest_separator  = #
18   FLIP:1    = filter = type == kayak
19   FLIP:1    = component = X -> xpos
20   FLIP:1    = component = Y -> ypos
21 }

```

The two echo mappings in lines 10 and 11 may be commented out to demonstrate the detection of echo mapping cycles. The pairs of mappings in Lines 6-7 and 8-9 demonstrate that a single incoming variable may be mapped to multiple destinations.

pDeadManPost: Arranging Posts in the Absence of Events

June 2018

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139
project-pavlab/appdocs/app_pdeadmanpost

1 Overview	1
2 Configuration Parameters for pDeadManPost	1
3 Publications and Subscriptions of pDeadManPost	2
3.1 Variables Published by pDeadManPost	2
3.2 Variables Subscribed for by pDeadManPost	2
4 Configuring the Heartbeat Condition	2
5 Specifying DeadFlags	3
6 Terminal and AppCast Output	4
7 A Simple Example	5

1 Overview

The `pDeadManPost` application allows the user to arrange configured posts to the MOOSDB in the *absence* of another event. There are many conceivable uses for this, but here are a few of the ideas that motivated this app:

- On a shoreside community, a dead-man post can be made to trigger an alert when a deployed vehicle is out of contact after some period of time. The alert could be a posting to trigger an alarm .wav file or a spoken alert message through `iSay`.
- On a surface vehicle, a dead-man post can be made to put the vehicle in a station-keeping mode if comms to the shoreside command and control goes silent for some period of time.
- On an underwater vehicle, a dead-man post can be made to put the vehicle in a return-to-home mode if it loses updates from navigation beacons for some period of time.

2 Configuration Parameters for pDeadManPost

The following parameters are defined for `pDeadManPost`. A more detailed description is provided in other parts of this section. Parameters having default values are indicated.

Listing 2.1: Configuration Parameters for `pDeadManPost`.

<code>heartbeat_var</code> :	Names the MOOS variable serving as the heartbeat. Section 4.
<code>active_at_start</code> :	if true then the heartbeat condition (timer) is active upon startup of the app. Otherwise the heartbeat condition is not active until the first heartbeat is received. The default is heartbeat true. Section 4.
<code>deadflag</code> :	A MOOS variable-value pair to be posted when the heartbeat disappears. Section 5.
<code>max_noheart</code> :	The maximum amount of time, in seconds, a missing heartbeat is tolerated before the deadflags are posted. Default is 10. Section 4.
<code>post_policy</code> :	Either "once", "repeat", or "reset" determines how the deadflags are posted once a heartbeat condition has failed. Section 5.

3 Publications and Subscriptions of pDeadManPost

The interface for `pDeadManPost`, in terms of publications and subscriptions, is described below. This same information may also be obtained from the terminal with:

```
$ pDeadManPost --interface or -i
```

3.1 Variables Published by pDeadManPost

The only output of `pDeadManPost` is:

- **APPCAST**: Contains an appcast report identical to the terminal output. Appcasts are posted only after an appcast request is received from an appcast viewing utility. Section 6.

The `pDeadManPost` app will also publish whatever MOOS variables are identified in the `deadflag` parameter.

3.2 Variables Subscribed for by pDeadManPost

The `pDeadManPost` application will subscribe for the following four MOOS variables:

- **APPCAST_REQ**: A request to generate and post a new appppcast report, with reporting criteria, and expiration. See the documentation on appcasting.

It will also subscribe for whatever MOOS variable is identified as serving as the heartbeat variable in the `heartbeat_var` parameter.

4 Configuring the Heartbeat Condition

The *heartbeat condition* stipulates that a named MOOS variable must be continually written to by some other application, with a specified frequency. If the heartbeat condition fails, deadflags are posted. The heartbeat is monitored simply by monitoring the mailbox in `pDeadManPost` for incoming mail on the named MOOS variable. No check is made as to the value or source of the incoming

mail, only that mail has been received. The monitored variable is set by `heartbeat_var` parameter. For example:

```
heartbeat_var = CONTINUE_MISSION
```

The frequency at which the heartbeat must be received is determined by `max_noheart` parameter, given in seconds. The default for this parameter is 10 seconds. It can be set, for example, with:

```
max_noheart = 120
```

The user may further specify whether the heartbeat condition is active immediately upon startup of the `pDeadManPost` app, or if it instead becomes active only after receiving the first heartbeat message. This is determined with the `active_on_start` parameter. For example:

```
active_on_start = false    // The default is true
```

The status of the heartbeat condition can be monitored in the appcasting output of the app, by noting the line "Time Remaining:". If this is zero, then the heartbeat condition has failed. If it reads "N/A", it means the heartbeat condition is not in effect, likely because the app is configured to be not active on start and no heartbeat message has yet been received. Otherwise, it should show the time remaining in seconds before the heartbeat condition fails unless another heartbeat message is received in the meanwhile.

5 Specifying DeadFlags

A `deadflag` is a MOOS variable-value pair to be posted when or if the heartbeat condition times out. Multiple dead flags may be used. An example:

```
deadflag = RETURN=true  
deadflag = DEAD_MAN_POST_INTERRUPT=true
```

When the heartbeat condition fails, and one or more deadman posts are made, by default, these posts are made only *once*, regardless of whether further heartbeat messages are received. The user has two further options: the posts can be made to *repeat* on each iteration of the `pDeadManPost` app by setting:

```
post_policy = repeat    // The default is "once"
```

The second option is to configure `pDeadManPost` to `reset` after a posting is made. In this case the heartbeat timer is reset and if another period of time elapses without a heartbeat, another posting is made. This is configured with:

```
post_policy = reset    // The default is "once"
```

When the `post_policy` parameter is set to `repeat`, the postings will cease when a new heartbeat message is received.

6 Terminal and AppCast Output

The `pDeadManPost` application produces some useful information to the terminal on every iteration of the application. An example is shown in Listing 2 below. This application is also appcast enabled, meaning its reports are published to the MOOSDB and viewable from any `uMAC` application or `pMarineViewer`. See the documentation on `uMac` or `uMacView` for more on appcasting and viewing appcasts. The counter on the end of line 2 is incremented on each iteration of `pDeadManPost`, and serves a bit as a heartbeat indicator. The "0/0" also on line 2 indicates there are no configuration or run warnings detected.

The output in the below example comes from the example described in Section 7.

Listing 6.2: Example terminal or appcast output for `pDeadManPost`.

```
1  =====
2  pDeadManPost alpha                                0/0(362)
3  =====
4  Configuration:
5  -----
6      heart_var: CONTINUE
7      max_noheart: 60
8  active_at_start: false
9      post_policy: repeat
10     deadflags: (1)
11         [1] RETURN=true
12
13  State:
14  -----
15     Elapsed Time: 23.0875
16  Time Remaining: 36.9125
17     Heartbeats: 2
18  Total Postings: 0
```

The first few lines (4-11) show the configuration settings for `pDeadManPost`. The status of `pDeadManPost` is shown in Lines 13-18. In this case, the most recent heartbeat message was received 23 seconds prior, as shown in line 15, and no deadman posts have been made as shown on line 18.

7 A Simple Example

The `s1_alpha_deadman` example mission distributed with `moos-ivp` provides a simple working example. More explanation to come...